MLOps Face Recognition System

Test Plan & Test Cases

Document Type:	Test Plan & Test Cases
System:	Siamese Face Recognition — Full MLOps Pipeline
Components:	Flask API · Trainer · MLflow · Optuna · PyTorch
Technology:	pytest · unittest.mock · SQLite in-memory
Date:	April 2026

1 Introduction & Purpose

This document constitutes the complete **Test Plan and Test Cases** for the MLOps Siamese Face Recognition system. It covers the testing strategy, environment setup, test scope, and the full suite of unit and integration tests implemented across two test files: **test_model.py** (model, data pipeline, configuration) and **test_api.py** (Flask REST API endpoints).

The test suite is designed to run on every Git push via **GitHub Actions CI**, requiring no GPU, no real dataset, and no running external services. All heavy dependencies — MLflow, the camera, and model loading — are mocked or replaced with in-memory equivalents, ensuring tests are fast, deterministic, and fully isolated.

1.1 Testing Objectives

Objective	Description
Correctness	Verify that model architecture, loss functions, and data utilities behave exactly as designed.
Isolation	Each test is fully independent — no test relies on state from another.
Speed	Full suite completes in under 60 seconds on CPU-only CI hardware.
Coverage	All critical code paths: model forward pass, loss mining variants, samplers, API endpoints.
Regression	Any future refactoring that breaks existing behaviour is immediately caught.

1.2 Test Files & Scope

File	Type	Covers
`test_model.py`	Unit Tests	Model architecture, triplet loss, config validation, PKSampler, data splits, register schema
`test_api.py`	Integration Tests	All 16 Flask REST endpoints via test client with mocked MLflow, camera, and in-memory SQLite DB
`conftest.py`	Configuration	Shared fixtures, pytest markers (slow, gpu), auto-skip for GPU tests when CUDA unavailable

2 Test Environment & Configuration

Tests are executed in a hermetically sealed environment. No network access, no GPU, no real dataset, and no running MLflow server are required. The following environment variables are set before any imports, ensuring consistent behaviour across local development and CI.

2.1 Environment Variables

Variable	Test Value	Purpose
`MLFLOW_TRACKING_URI`	`sqlite:///test_mlflow.db`	Redirects MLflow to a local SQLite file
`DATASET_NAME`	`lfw`	Selects the LFW identity dataset

Variable	Test Value	Purpose
DB_PATH	:memory:	In-memory SQLite for API tests — no disk I/O
MLFLOW_MODEL_NAME	TestModel	Prevents interference with production registry
MISCLASSIFY_THRESHOLD	5	Threshold for misclassification-triggered retrain

2.2 Mocking Strategy

The Flask API tests use **unittest.mock.patch** to replace external dependencies at import time. This prevents any real network call, camera access, or model load during testing.

Patched Target	Reason
app.load_best_model_from_mlflow	Prevents MLflow model download on startup
app.threading.Thread	Prevents background camera thread from starting
app.cv2.VideoCapture	Prevents camera initialisation on CI hardware
SQLite (DB_PATH=:memory:)	Provides an isolated, ephemeral database per test session

2.3 conftest.py — Shared Fixtures & Markers

The **conftest.py** file is loaded automatically by pytest before any test collection. It performs three tasks:

- Inserts **src/** onto sys.path so all test modules can import project code without installing the package.
- Registers custom markers — slow (for long-running tests) and gpu (for CUDA-requiring tests) — so pytest does not emit unknown-marker warnings.
- Auto-skips any test marked @pytest.mark.gpu when torch.cuda.is_available() returns False, making the suite pass on CPU-only CI runners without modification.

3 Model & Pipeline Unit Tests (test_model.py)

All tests in this file use **synthetic tensors** — randomly generated with torch.randn() or manually crafted arrays — and run entirely on CPU. No dataset download is required. The file is organised into six test classes, each targeting a specific subsystem.

3.1 TestSiameseEncoder — Model Architecture

Verifies that the **SiameseNetwork** (model.py) is constructed correctly and that its encoder produces geometrically valid embeddings. The encoder is a ResNet50 backbone (frozen) followed by a trainable projection head that maps features to a 128-dimensional L2-normalised embedding space.

ID	Test Name	How It Runs	Expected Result	Status
TC-M-01	test_build_model_returns_siamese_network	Call build_model(); check isinstance(model, SiameseNetwork)	Returns a SiameseNetwork instance	PASS
TC-M-02	test_encoder_output_shape	Feed batch of 4 images (3×224×224); check embedding shape	Shape is (4, 128) matching EMBEDDING_DIM	PASS
TC-M-03	test_embeddings_are_l2_normalized	Compute embeddings for 4 synthetic images; check unit norms	torch.norm(emb, dim=1) ≈ 1.0 (atol=1e-5)	PASS
TC-M-04	test_forward_returns_distance	Pass two batches of 2 images through model.forward()	Returns (emb1, emb2, dist); dist.shape==(2,); dist>=0	PASS
TC-M-05	test_backbone_frozen_by_default	Check requires_grad on all backbone parameters	All backbone params have requires_grad=False	PASS
TC-M-06	test_head_is_trainable	Check requires_grad on embed layer parameters	At least one embed param has requires_grad=True	PASS
TC-M-07	test_same_input_zero_distance	Pass identical image tensor as both inputs to forward()	Euclidean distance < 1e-4	PASS

Design note: TC-M-03 is critical — the cosine similarity matcher in the inference engine assumes unit-norm embeddings. If L2 normalisation ever breaks, recognition scores become meaningless. TC-M-05 verifies Phase 1 training (backbone frozen, only head trains) is correctly enforced by build_model().

3.2 TestTripletLoss — Loss Function

Validates the **TripletLoss** implementation in utils.py across all three mining strategies: **all** (all valid triplets), **hard** (hardest positive/negative per anchor), and **semi** (semi-hard negatives — negatives further than the positive but within margin). The fixture creates 8 embeddings from 4 identities, 2 samples each.

ID	Test Name	How It Runs	Expected Result	Status
TC-L-01	test_loss_is_non_negative[all]	Compute loss on 8 synthetic embeddings, 4 classes, mining='all'	loss.item() >= 0	PASS
TC-L-02	test_loss_is_non_negative[hard]	Same fixture, mining='hard'	loss.item() >= 0	PASS
TC-L-03	test_loss_is_non_negative[semi]	Same fixture, mining='semi'	loss.item() >= 0	PASS
TC-L-04	test_loss_returns_n_active [all/hard/semi]	Check second return value of forward()	n_active is int and >= 0	PASS

ID	Test Name	How It Runs	Expected Result	Status
TC-L-05	test_invalid_mining_raises	Instantiate TripletLoss(mining='invalid_strategy'); call forward()	Raises ValueError matching 'Unknown mining'	PASS
TC-L-06	test_perfect_embeddings_have_low_loss	3 classes, each at orthogonal unit vector; margin=0.3, mining='hard'	loss < 0.01 — perfectly separated embeddings should have ~0 loss	PASS

Design note: TC-L-05 guards against silent failures if a new mining strategy key is introduced with a typo. TC-L-06 provides a sanity check that the loss arithmetic is directionally correct — loss should collapse to near zero for maximally separated embeddings.

3.3 TestConfig — Configuration Validation

These lightweight tests assert that config.py exposes numerically sane defaults. They run in milliseconds and catch any accidental zero-initialisation or out-of-range value introduced by environment variable overrides.

ID	Test Name	How It Runs	Expected Result	Status
TC-C-01	test_embedding_dim_positive	Assert config.EMBEDDING_DIM > 0	EMBEDDING_DIM is 128 (positive)	PASS
TC-C-02	test_train_val_ratios_sum_lt_one	Assert TRAIN_RATIO + VAL_RATIO < 1.0	0.70 + 0.15 = 0.85 < 1.0 — test split remains	PASS
TC-C-03	test_batch_size_divisible_by_4	Assert BATCH_SIZE % 4 == 0	BATCH_SIZE=128; 128 % 4 == 0	PASS

Design note: TC-C-03 is required because PKSampler asserts batch_size % k == 0 internally. If BATCH_SIZE is changed to a non-multiple of 4, the sampler would crash at runtime — this test surfaces the failure at config load time instead.

3.4 TestPKSampler — Batch Sampler

The **PKSampler** (dataloader.py) yields batches of exactly P×K samples. Each batch contains P randomly chosen identities, each represented by K images. This sampling strategy is essential for effective triplet mining — each batch is guaranteed to contain positive pairs.

ID	Test Name	How It Runs	Expected Result	Status
TC-P-01	test_batch_size_respected	Create PKSampler with 20 identities × 4 images, batch_size=8, n_batches=10; iterate all batches	Every yielded batch has exactly 8 indices	PASS
TC-P-02	test_batch_size_not_divisible_raises	Instantiate PKSampler(batch_size=7, k=4)	Raises AssertionError — 7 % 4 != 0	PASS
TC-P-03	test_n_batches_length	Create sampler with n_batches=5; check len() and full iteration count	len(sampler)==5 and list(sampler) yields exactly 5 batches	PASS

3.5 TestDataPrep — Data Splitting & Misclassified Merge

Verifies that **split_identities()** and **merge_misclassified_into_train()** in dataloader.py produce the correct identity-level partitions with no cross-contamination. TC-D-03 uses tmp_path (pytest's built-in temporary directory fixture) to create a realistic fake misclassified directory with tiny JPEG images.

ID	Test Name	How It Runs	Expected Result	Status
TC-D-01	test_split_identities_proportions	100 identities; ratios 0.70/0.15; seed=42	train=70, val=15, test=15 identities	PASS
TC-D-02	test_split_no_identity_overlap	50 identities; split; check set intersections	train∩val=∅, train∩test=∅, val∩test=∅	PASS
TC-D-03	test_merge_misclassified_adds_to_train_only	Create tmp dir with 'Alice' (existing) and 'NewPerson' (new); call merge on train_map	Alice gains 1 new crop; NewPerson is added; Bob unchanged; val and test untouched	PASS

Design note: TC-D-02 is the most important data integrity test. If identities leak between train and test sets, evaluation metrics become optimistic, invalidating model selection. TC-D-03 verifies the HITL feedback loop — misclassified crops must only augment the training set.

3.6 TestRegisterModel — Artefact Schema

Validates that the **best_run.json** artefact — written by both sweep_optuna.py and main.py — conforms to the schema expected by register_model.py. This is a contract test: it ensures the handshake between the sweep stage and the registration stage is never silently broken.

ID	Test Name	How It Runs	Expected Result	Status
TC-R-01	test_best_run_json_schema	Write a sample best_run dict to tmp_path; reload; check keys	Keys run_id, val_loss, params all present in loaded JSON	PASS

4 Flask API Integration Tests (test_api.py)

All API tests use **Flask's built-in test client** (app.test_client()), which simulates HTTP requests without running a real server. The test client shares the application context, allowing direct inspection of response status codes, JSON payloads, and headers. A module-scoped fixture client patches out all external dependencies at startup and provides a fresh in-memory SQLite database for the entire test session.

4.1 Test Client Fixture — How It Works

The @pytest.fixture(scope='module') client fixture performs the following setup sequence before any test in the module runs:

1. **Patch app.load_best_model_from_mlflow** — replaces the MLflow model loader with a no-op mock so no checkpoint download is attempted.
2. **Patch app.threading.Thread** — prevents the background camera-reading thread from starting.
3. **Patch app.cv2.VideoCapture** — prevents camera initialisation on headless CI.
4. **Set DB_PATH to a temporary file** — each test session gets a private SQLite file (deleted after), preventing state leakage between CI runs.
5. **Call init_db()** — creates the tables (faces, misclassifications) in the fresh database.
6. **Yield the Flask test client** — all tests in the module receive the same configured client.

4.2 TestHealthEndpoints — Service Health & Observability

These three tests verify the operational health endpoints that are scraped by Prometheus and used by the Airflow DAG to check if the Flask service is ready before triggering a reload.

ID	Test Name	Request	Expected Result	Status
TC-A-01	`test_health_returns_200`	`GET /health`	HTTP 200	PASS
TC-A-02	`test_ready_returns_json`	`GET /ready`	HTTP 200 or 503; response body contains 'status' key (503 is acceptable when no model is loaded)	PASS
TC-A-03	`test_metrics_returns_prometheus_format`	`GET /metrics`	HTTP 200; body text contains 'requests_total', 'uptime_seconds', 'misclassifications_total'	PASS

Design note: TC-A-03 checks that the Prometheus exposition format is correctly produced. If the metric names change, Grafana dashboards and alert rules would break silently — this test prevents such regressions.

4.3 TestPeopleAPI — Face Registration Management

Tests the CRUD-like endpoints for managing registered face identities. Because the database is initialised empty for each test session, the 'list' test asserts an empty array initially.

ID	Test Name	Request	Expected Result	Status
TC-A-04	`test_list_people_empty_initially`	`GET /api/people`	HTTP 200; body is an empty JSON array []	PASS
TC-A-05	`test_delete_nonexistent_person_ok`	`POST /api/delete {"name": "nobody"}`	HTTP 200 — deleting a non-existent person is idempotent	PASS
TC-A-06	`test_delete_missing_name_fails`	`POST /api/delete {}`	HTTP 400 — missing required 'name' field returns Bad Request	PASS

4.4 TestMisclassificationAPI — HITL Feedback Loop

The misclassification reporting endpoints are the entry point of the Human-in-the-Loop (HITL) feedback loop. When the system misidentifies a person, the user can report the correct label via the UI. If the count exceeds a threshold, the

DAG is triggered automatically. These tests verify the full path: input validation → persistence → stats retrieval.

ID	Test Name	Request	Expected Result	Status
TC-A-07	test_report_misclassification_requires_true_name	POST /api/report_misclassification {"predicted": "Bob", "score": 0.3}	HTTP 400 — true_name is mandatory; request missing it is rejected	PASS
TC-A-08	test_report_misclassification_logs_correctly	POST /api/report_misclassification {"true_name":"Alice", "predicted":"Bob", "score":0.45}	HTTP 200; {ok:true, stats:{total>=1}}	PASS
TC-A-09	test_misclassification_stats_endpoint	GET /api/misclassification_stats	HTTP 200; body contains 'total', 'unique_pending', 'threshold' keys	PASS
TC-A-10	test_flag_retrain_requires_true_label	POST /api/flag_retrain {}	HTTP 400 — true_label is mandatory for retrain flagging	PASS

Design note: TC-A-08 directly exercises the SQLite write path — after reporting a misclassification, stats.total must be >= 1, confirming the record was persisted. TC-A-10 protects the retrain trigger endpoint from being fired without a validated label.

4.5 TestRegisterAPI — Face Registration Workflow

The registration API sets the Flask application into 'register mode', causing the video loop to capture the next face and store its embedding. These tests verify input validation and state transitions without triggering any camera activity.

ID	Test Name	Request	Expected Result	Status
TC-A-11	test_register_requires_name	POST /api/register {}	HTTP 400 — name field is required	PASS
TC-A-12	test_register_sets_mode	POST /api/register {"name": "Alice"}	HTTP 200; {ok: true} — arms the camera loop for face capture	PASS
TC-A-13	test_cancel_register	POST /api/cancel_register {}	HTTP 200 — registration mode is cancelled	PASS

4.6 TestStatusAPI — Application State Introspection

The status endpoints expose the internal state of the Flask application — current mode, last recognition result, active model version, and misclassification counters. They are polled by the frontend UI to update the display and by the Airflow notify task to confirm successful model reload.

ID	Test Name	Request	Expected Result	Status
TC-A-14	test_status_returns_expected_keys	GET /api/status	HTTP 200; body contains all of: 'mode', 'last_result', 'model_version', 'misclassify'	PASS
TC-A-15	test_model_info_endpoint	GET /api/model_info	HTTP 200; body contains 'model_name' and 'threshold'	PASS

5 Test Execution & CI Integration

5.1 Running Tests Locally

Install dependencies and execute the full suite from the project root:

```
# Install test dependencies pip install pytest torch torchvision flask flask-cors pillow # Run all tests pytest tests/ -v # Run only model tests (fast, no Flask) pytest tests/test_model.py -v # Run only API tests pytest tests/test_api.py -v # Skip slow tests pytest tests/ -v -m 'not slow'
```

5.2 GitHub Actions CI Pipeline

Tests are automatically triggered on every push and pull request. The CI workflow runs on ubuntu-latest with Python 3.10, installs all dependencies from requirements-test.txt, and fails the build if any test does not pass. GPU-marked tests are automatically skipped because GitHub-hosted runners have no CUDA device.

Property	Value
Trigger	Every push and pull_request to any branch
Runner	ubuntu-latest (GitHub-hosted, CPU only)
Python	3.10
GPU tests	Auto-skipped via conftest.py (torch.cuda.is_available() == False)
Expected time	< 60 seconds for full suite on CPU
Failure policy	Any test failure blocks merge — branch protection enforced

6 Test Coverage Summary

The table below provides a consolidated view of all 21 test cases across both test files, mapping each test class to the source module it covers and the overall verdict.

Test Class	File	Module Covered	Tests	Pass	Fail
TestSiameseEncoder	test_model.py	model.py	7	7	0
TestTripletLoss	test_model.py	utils.py	6	6	0
TestConfig	test_model.py	config.py	3	3	0
TestPKSampler	test_model.py	dataloader.py	3	3	0

Test Class	File	Module Covered	Tests	Pass	Fail
TestDataPrep	test_model.py	dataloader.py	3	3	0
TestRegisterModel	test_model.py	register_model.py (schema)	1	1	0
TestHealthEndpoints	test_api.py	app.py (/health /ready /metrics)	3	3	0
TestPeopleAPI	test_api.py	app.py (/api/people /api/delete)	3	3	0
TestMisclassificationAPI	test_api.py	app.py (misclassification loop)	4	4	0
TestRegisterAPI	test_api.py	app.py (/api/register)	3	3	0
TestStatusAPI	test_api.py	app.py (/api/status /api/model_info)	2	2	0
TOTAL	—	—	**38**	**38**	**0**

Note: The parametrised TripletLoss tests (mining=all/hard/semi) are counted once each for non-negative loss and n_active — they expand to 6 test runs via @pytest.mark.parametrize but represent 2 logical test cases per variant.

7 Testing Design Rationale

7.1 Why Synthetic Data Instead of Real Images?

The LFW dataset is approximately 250 MB and is not available on CI runners. Downloading it during every test run would be slow, costly, and fragile (network dependency). Synthetic torch.randn() tensors exercise exactly the same code paths — the model does not distinguish between real faces and random noise for the purposes of shape, norm, and distance assertions. This approach makes the test suite fully self-contained.

7.2 Why Module-Scoped Fixtures for the API Tests?

Creating the Flask test client is relatively expensive — it involves patching three dependencies, creating a SQLite file, and initialising the database schema. Using scope='module' means this setup runs once per test file, not once per test. Each test within the module receives the same client. Tests that write to the database (TC-A-08) do not reset between tests — this is intentional: TC-A-09 reads the stats that TC-A-08 wrote, validating end-to-end persistence in sequence.

7.3 Why Test the Prometheus Output Format?

Prometheus metric names are strings scraped by an external system. If a developer renames requests_total to request_count, Grafana dashboards silently stop displaying data and alert rules stop firing. TC-A-03 pins the metric names as a contract test, making any such rename visible as an immediate CI failure.

7.4 Why Test TC-L-06 (Perfect Embeddings)?

A triplet loss implementation can be numerically correct yet directionally wrong — for example, if the d_ap and d_an terms are accidentally swapped, the loss would still be non-negative (TC-L-01 would pass) but the model would learn to push same-class embeddings apart. TC-L-06 constructs a scenario where the answer is known analytically (loss ≈ 0) and verifies that the implementation produces that answer, catching directional bugs that non-negativity alone cannot detect.

8 Test Scorecard

Category	Test IDs	Passed / Total	Result
Model Architecture	TC-M-01 to TC-M-07	7 / 7	PASS
Triplet Loss	TC-L-01 to TC-L-06	6 / 6	PASS
Configuration	TC-C-01 to TC-C-03	3 / 3	PASS
PKSampler	TC-P-01 to TC-P-03	3 / 3	PASS
Data Pipeline	TC-D-01 to TC-D-03	3 / 3	PASS
Artefact Schema	TC-R-01	1 / 1	PASS
Health Endpoints	TC-A-01 to TC-A-03	3 / 3	PASS
People API	TC-A-04 to TC-A-06	3 / 3	PASS
Misclassification	TC-A-07 to TC-A-10	4 / 4	PASS
Register API	TC-A-11 to TC-A-13	3 / 3	PASS
Status API	TC-A-14 to TC-A-15	2 / 2	PASS
OVERALL	TC-M-01 → TC-A-15	**38 / 38**	ALL PASS

All 38 test cases pass on CPU-only hardware with no external services. The suite is suitable for GitHub Actions CI and forms part of the complete Design & Implementation Review submission.